Ders 13 Çalışma Özeti

Ders 13 Çalışma Özeti

Genel Konular

- Bütüncül refactoring örneği: video kiralama uygulaması
  - Önceki mevcut derste (ders 11) bitirilen kod kusurları ve refactoring eylemleri, Fowler'ın Refactoring kitabının 1. bölümündeki video kiralama örneği üzerinde baştan sona uygulandı. Kod en kötü halinden başlayıp adım adım iyileştirildi; her adımın kodu Eclipse'te ayrı bir pakette (`fowler_00`, `fowler_01`, …, `fowler_09`, `fowler_10_alt`) tutuldu ve paketler Eclipse'in karşılaştırma aracıyla yan yana gösterildi (ekrandan, 00:10:00).
  - Sınıflar: `Customer` (`_name`, `Vector<Rental> _rentals`, `addRental`, `getName`, `statement`), `Rental` (`_daysRented`, `_movie`), `Movie` (`_title`, `_priceCode`, `REGULAR = 0`, `NEW_RELEASE = 1`, `CHILDRENS = 2`). Bir müşterinin birçok kira kaydı, her kira kaydının bir filmi vardır.
  - Başlangıçtaki sorun: `Customer.statement()` hocanın deyişiyle 40 satıra yakın bir "kusurlar yumağı"dır. İçinde fiyat koduna göre `switch`, geçici değişkenler, bonus hesabı, makbuz satırı basımı ve toplamlar bir aradadır (ekrandan, 00:09:55 ve 00:20:00).
- Başlangıç kodundaki iş kuralları (ekrandan, 00:20:00 ve 00:10:00)
  - `REGULAR`: 2.0; kira günü 2'den fazlaysa `(gün - 2) * 1.5` eklenir.
  - `NEW_RELEASE`: `gün * 3.0`.
  - `CHILDRENS`: 1.5; kira günü 3'ten fazlaysa `(gün - 3) * 1.5` eklenir.
  - Bonus (frequent renter points): her kiralama 1 puan; yeni çıkan film 1 günden uzun kiralandıysa 1 puan daha.
  - Makbuz: "Rental Record for ", her film için başlık ve bedel, sonda "Amount owed is …" ve "You earned … frequent renter points".
- Adım adım düzenlemeler
  - 1. adım, Extract Method: `switch` bloğu `private double amountFor(Rental each)` metoduna çıkarıldı; `statement` içinde `thisAmount = amountFor(each);` kaldı (ekrandan, 00:21:20).
  - 2. adım, değişken adlarını düzenleme: `amountFor` artık tek bir kira kaydının bedelini hesapladığı için `each` adı `aRental` yapıldı; metot içindeki `thisAmount` de `result` oldu.
  - 3. adım, Move Method: `amountFor` müşterinin hiçbir verisini kullanmıyor, yalnız kira kaydı ve filmle ilgili veri kullanıyor; bu yüzden `Rental`'a taşındı ve adı `getCharge()` oldu. `statement` içinde `thisAmount = aRental.getCharge();` (ekrandan, 00:35:10).
  - 4. adım, Replace Temp with Query: `thisAmount` bir kez atanıp hiç değişmediği için kaldırıldı, kullanıldığı iki yerde `aRental.getCharge()` çağrıldı (ekrandan, 00:40:00).
  - 5. adım, bonus için Extract Method + Move Method: bonus hesabı `Rental.getBonus()` metoduna taşındı; geçici değişken artırmak yerine koşul doğruysa `return 2;`, değilse `return 1;` (ekrandan, 00:44:45). "frequent renter points" yerine daha kısa ve anlaşılır bulunan "bonus" adı kullanıldı.
  - 6. adım, toplamlar için yeni sorgu metotları: `Customer.getTotalCharge()` ve `Customer.getTotalBonus()` eklendi; `statement` içindeki `totalAmount` ve `bonus` geçici değişkenleri kalktı, makbuzun sonunda bu metotlar çağrıldı (ekrandan, 00:48:50).
  - 7. adım, bedel hesabı filme taşındı: `Movie.getCharge(int daysRented);` `Rental.getCharge()` yalnız `return _movie.getCharge(_daysRented);` yapar (ekran-

dan, 00:57:30 ve 00:58:05).
  - 8. adım, bonus hesabı da filme taşındı: `Movie.getBonus(int daysRented)`; `Rental.getBonus()` çağrıyı iletir (ekrandan, 01:00:10).
  - 9. adım, `Rental.getMovieTitle()` eklendi; `Customer` artık `aRental.getMovie().getTitle()` yerine `aRental.getMovieTitle()` çağırır (ekrandan, 01:05:58).
  - 10. adım (slayt 64): `switch` ve `if` yerine çok biçimlilik: soyut `Movie` ve `MovieNewRelease`, `MovieRegular`, `MovieChildrens` alt sınıfları (ekrandan, 01:07:50).
  - Alternatif 10. adım (`fowler_10_alt`): kalıtım yerine sahiplik. `Movie` bir `Price _price` üyesi tutar; soyut `Price` ve `PriceRegular`, `PriceNewRelease`, `PriceChildrens` alt sınıfları hesaplamayı yapar (ekrandan, 01:16:15 ve 01:17:35).
- Testler ve kara kutu / beyaz kutu yaklaşımı
  - Her adımdan sonra JUnit testleri (`TestAll`, `TestCustomer`, `TestMovie`) çalıştırıldı. Arayüz (dışa açılan API) değişmediği için önceki paketin test kodu yeni pakete aynen kopyalanıp çalıştırılabildi.
  - Kara kutu yaklaşımında yeni metot (`Rental.getCharge`) için yeni test gerekmez; beyaz kutu yaklaşımında iç yapıya göre yeni test eklemek gerekir.
- Etkileşim (sequence) şeması üzerinden bağlaşım
  - 8. adımın etkileşim şemasında `Customer`'dan `Movie`'ye doğrudan giden tek çağrı `getTitle()`'dır (ekrandan, 01:02:50). Bu "çıkıntı" 9. adımda kaldırılınca `Customer` yalnız `Rental` ile, `Rental` da `Movie` ile konuşur; katmanlı bir ilişki kurulur ve bağlaşım azalır.
- Anti-patterns (karşıt kalıplar)
  - Karşıt kalıp, bir tasarım kalıbı gibi görünen ama kötü modellemeye götüren çözümdür; yazılım geliştirmede tekrar eden yanlışları adlandırır (ekrandan, 01:22:30).
  - Kod kusurları gerçekleme (kod) düzeyine, karşıt kalıplar tasarım düzeyine daha yakındır. Karşıt kalıplar yazılım yaşam döngüsünün diğer evreleri ve proje yönetimiyle de ilişkilidir.
  - Üç grup: yazılım geliştirme, yazılım mimarisi ve yazılım proje yönetimi karşıt kalıpları.

Hocanın Özellikle Vurguladığı Kısımlar

- Refactoring'in özü dışarıya açılan hizmet penceresini değiştirmeden iç yapıyı düzenlemektir.
  - Bu yüzden testler hiç değişmeden her adımda geçti. "Arayüz" burada Java `interface`'i değil, API anlamındadır.
- Geriye doğru uyumluluk ile ideal tasarım arasında seçim yapmak gerekebilir.
  - Sıfırdan tasarımda film türleri `Movie`'nin alt sınıfları olurdu; kurucuya fiyat kodu vermenin anlamı kalmazdı. Ama `Movie` kurucusundaki fiyat kodu ve public sabitler testlerde de kullanıldığı için bu tasarıma geçmek geriye uyumluluğu bozar. Uyumluluk önemliyse orada durulur, değilse devam edilir.
- Bir işin sorumluluğu, o işle ilgili veriye en çok sahip olan sınıfa verilir; bölünebiliyorsa her parça kendi verisinin sahibine gider.
  - Bedel, kira süresine (`Rental`) ve film türüne (`Movie`) bağlıdır. Film türüne bağlı kısım `Movie`'ye taşınır, `Rental` kira süresini parametre olarak verir.
- Küçük adımlarla ilerlenir, her adımdan sonra testler çalıştırılır.
  - Bir refactoring adımı yeni bir kod kusuru doğurabilir; en göze batan yerden başlanır ve sonuçtan memnun olunana kadar zincirleme düzeltilir.
- Eclipse'in otomatik refactoring araçlarına tek başına güvenilmemeli.
  - Hocanın okuduğu eski bir makaleye göre bu araçlar yaklaşık %70-80 oranında hatasızdır; hoca ders notlarını hazırlarken Extract Method'un geçici değişkenler yüzünden başarısız olduğunu gördü. Kullanılırsa testler mutlaka yeniden çalıştırılır.
- `switch-case` nesneye dayalı tasarımda en çok göze batan yapıdır.
  - Kalıtım ve çok biçimlilikle ya da sahiplikle (Price) ortadan kaldırılır.
- Toplamlar için listeyi üç kez dolaşmak kayda değer bir performans kaybı yaratmaz.
  - Liste milyonlarca kayıt içerse ya da ağ üzerinden gelse bile sistemin asıl darboğazı büyük olasılıkla burası değildir.

Kısa Tekrar Notları

- Video kiralama örneği: `Customer` 1 → * `Rental` * → 1 `Movie`; kötü kodun merkezi `Customer.statement()`.
- Sıra: Extract Method (`amountFor`) → yeniden adlandırma (`each` → `aRental`) → Move Method (`Rental.getCharge`) → Replace Temp with Query (`thisAmount`) → bonus için Extract + Move (`Rental.getBonus`) → `getTotalCharge` / `getTotalBonus` → `Movie.getCharge(int)` → `Movie.getBonus(int)` → `Rental.getMovieTitle()` → çok biçimlilik ya da `Price` sınıfları.
- Kümülatif toplam tutan geçici değişken tamamen kaldırılamaz; ama kısa bir sorgu metodunun içine alınabilir.
- Arayüz değişmezse aynı test kodu her adımda kullanılır.
- `Customer`'dan `Movie`'ye giden doğrudan çağrı kalktı: bağlaşım azaldı; `Rental`'a sorumluluk taşınınca `Customer`'ın uyumu yükseldi.
- `Price` soyut sınıfı: `getCharge(int)` ve `getPriceCode()` soyut, `getBonus(int)` varsayılan olarak 1 döndürür; `PriceNewRelease` onu ezer.
- Karşıt kalıplar: tasarım düzeyi; üç grup (geliştirme, mimari, proje yönetimi).

Detaylı Açıklamalar

- Neden bu sıra?
  - Hoca kitaptaki sırayı izlediğini, kendi aklına gelen sırayla ilerlerse notlardan sapabileceğini söyledi. İlk hedef en göze batan şeydir: `statement` metodunun devasa olması ve içindeki `switch`. Önce bu blok `amountFor` adlı bir metoda çıkarıldı. Metot dışarıya hizmet vermediği için `private` tanımlandı.
  - Ardından değişken adları düzeltildi. `each` adı liste içinde dolaşırken anlamlıydı; tek bir kiranın bedelini hesaplayan metotta anlamsızdır. Adlandırma da Eclipse'in Refactor menüsünde yer alan bir düzenlemedir; anlaşılırlığı artırır.
- Move Method ve uyum
  - `amountFor` içinde müşteriyle ilgili hiçbir veri yoktur: filmin fiyat kodu ve kira günü kullanılır. Bu yüzden metot `Rental`'a taşındı ve adı `getCharge` oldu ("ey kira kaydı, ödemem gereken bedeli bana getir"). `Rental` içinde `getMovie()` ve `getDaysRented()` yerine doğrudan kendi üyeleri kullanılabilir. Hoca metodu önce paket erişimli bıraktığını (Java'da erişim belirteci yazılmazsa paket erişimi olur), sonraki pakette `public` yaptığını söyledi.
  - Bu taşıma, hangi metodun hangi üyeye eriştiğine bakan statik bağlaşım ve uyum ölçülerini de iyileştirir; ama asıl kazanç anlamsaldır: sınıfa ait olmayan bir sorumluluk ait olduğu yere gitmiştir.
- Geçici değişkenler
  - `thisAmount` bir kez değer alıp değişmediği için Replace Temp with Query ile kaldırıldı; metot iki kez çağrılır ama döngüde değildir ve içi hafiftir. `totalAmount` ve bonus toplamı ise kümülatiftir, bir yerde biriktirilmesi gerekir; bunlar `getTotalCharge()` ve `getTotalBonus()` içinde `result` adlı yerel değişkenlere alındı. Hoca geçici değişkenin asıl zararının metodu uzatma eğilimi olduğunu, kısa ve odaklı bir metot içindeki geçici değişkenin sorun olmadığını söyledi.
  - `getBonus`'ta "önce 1 artır, koşul doğruysa 1 daha artır" yerine "koşul doğruysa 2, değilse 1 döndür" yazılarak geçici değişkene gerek kalmadı.
  - Notlardaki öneriye göre `getTotalCharge` ve `getTotalBonus` içindeki `result` adları `totalCharge` ve `totalBonus` yapılabilir; hoca bunu küçük bir iyileştirme saydı ve yapmadı.
- Hesabın filme taşınması
  - `Rental.getCharge()` içinde `switch` hâlâ başka bir nesnenin (filmin) fiyat koduna göre karar veriyordu. Bir nesnenin verisiyle ilgili kararlar o nesnede verilmelidir. Fiyat koduna bağlı kısım `Movie.getCharge(int daysRented)` metoduna taşındı; kira süresi `Rental`'da kaldı ve parametre olarak geçirildi: "seni 3 gün kiraladılar, buna göre ne kadar ödenmesi gerektiğini hesapla". `switch` hâlâ durur, ama artık sınıfın kendi verisi üzerindedir.
- Etkileşim şeması ve `getMovieTitle`
  - Hoca 1. adımdan beri slaytlarda gösterilen etkileşim şemasının ileride bir şeyi fark ettireceğini söylemişti. 8. adımın şemasında `Customer` her şeyi `Rental` üzerinden yapar, tek istisna film başlığını almak için `Movie`'ye yaptığı doğrudan `getTitle()` çağrısıdır.

`Rental`'a `getMovieTitle()` eklenince bu çağrı kalkar. Sınıf şemasında görünmese de `Customer` ile `Movie` arasında dolaylı bir ilişki vardı; artık kaynak kodda da yoktur.

- İdeal tasarım ve `Price` çözümü
  - Hoca sıfırdan tasarlasa `getCharge` ve `getBonus`'un soyut bir `Movie` sınıfında soyut metot olarak duracağını, her film türünün bir alt sınıf olacağını söyledi. `Movie` arayüz (`interface`) olamaz, çünkü adı (title) ve gövdesi yazılabilen metotları vardır; bu yüzden soyut sınıf en uygunudur. Ancak bu tasarım geriye uyumluluğu bozar.
  - Kalıtım yerine sahiplik de kullanılabilir: filmlere farklı fiyat etiketleri yapıştırmak gibi. `Movie`'ye bir `Price` üyesi eklenir; `Price` soyut sınıfının alt sınıfları farklı fiyatlandırmaları gerçekler. Hoca bunun tam Strategy olmadığını ama Strategy'ye benzediğini, önemli olanın kalıp aklına gelmeden "nasıl daha iyi modellerim" diye düşünmek olduğunu söyledi.
  - Geriye uyumluluk için public sabitler ve kurucudaki fiyat kodu kalır. Kurucu `setPriceCode(priceCode)` çağırır; bu metot fiyat koduna göre `PriceChildrens`, `PriceNewRelease` ya da varsayılan olarak `PriceRegular` nesnesini oluşturur. Yani bir `switch` yine vardır, ama yalnız ilklendirme yapar; hesaplamalar artık `switch` içinde değildir. `getPriceCode()` artık `return _price.getPriceCode();` yapar (ekrandan, 01:20:00).
  - `Price` sınıfında `getBonus(int)` gövdeli yazıldı (varsayılan 1 puan), çünkü çoğu alt sınıf bu gövdeyi kalıtımla kullanabilir; yalnız `PriceNewRelease` bu metodu ezer: `daysRented > 1` ise 2, değilse 1 (ekrandan, 01:17:35).
- Karşıt kalıplar ve kaynaklar
  - Tasarım kalıpları iyi tasarımın nasıl yapıldığını, karşıt kalıplar kötü tasarımın nasıl yapıldığını gösterir. Hoca konuyla ilgilenenlere Fowler'ın kitabında derste görülmeyen kod kusurlarını, literatürdeki başka kod kusurlarını ve karşıt kalıpları önerdi. Kod kusurlarını kategorilere ayıran bir web sitesini (sourcemaking.com) gösterdi: Bloaters grubunda Long Method, Large Class, Primitive Obsession, Long Parameter List, Data Clumps (ekrandan, 01:40:00). Sitede "Spaghetti Code" gibi karşıt kalıp adları da vardır.
- Ders sonrası notlar
  - Hoca `Price.getBonus` açıklamasında (01:16:30) "kira günü 2'den büyükse 2 puan" dedi; ekrandaki kodda koşul `daysRented > 1`'dir (ekrandan, 01:17:35). Doğrusu: yeni çıkan film 1 günden uzun kiralanırsa 2 puan.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.